PROGRAMA · TRAIN THE TRAINERS

Formación *AI Engineering.*

Prompting, contexto, seguridad, agent skills, templates del SDLC, tool calling con MCP, optimización y casos E2E (Hackathon)

DURACIÓN	FORMATO	COBERTURA
8 módulos × 5 h · 40 h	**Trainers internos BCP**	**5 grupos × 5 personas**

© 2026 NTT DATA, Inc.

NTT DATA

1

quipos
de **desarrollo** de **software**

Modelo de niveles de madurez, *SDLC con GenIA.*

Este es el modelo de madurez en BCP, evolucionar el uso individual de IA hacia practicas estructuradas y optimas.

1 Vibe

Prompts libres.

Uso puntual de herramientas GenAI (Copilot). Adopción **no estandarizada**.

- Dependencia del conocimiento individual
- Resultados no trazables
- Conocimiento no reutilizable

Hoy · uso disperso

2 Vibe

Prompts guiados.

Pasamos del uso libre a patrones de prompt **alineados al estándar BCP**.

- Arquitectura y seguridad considerados
- CI/CD consciente
- Buenas prácticas aisladas

Tránsito · parcial

3 Spec

Agent skills.

Prompts convertidos en **skills versionadas y reutilizables**.

- Integración al SDLC y repos
- Gobierno y ownership definido
- Skills gobernadas en catálogo

Objetivo · programa de formación

4 Spec

Agentes autónomos.

Agentes que ejecutan flujos complejos **bajo reglas y supervisión**.

- Orquestación multi-agente
- Observabilidad y control operativo
- Gobierno maduro, monitoreado

Aspiración · escala

Vibe-Coding Development

De prompt suelto a prompt template.

Niveles 1 y 2. La persona dicta instrucciones y las modifica. La IA todavía **no tiene memoria de la decisión técnica**.

Spec-Driven Development

De skill versionada a agentes supervisados.

Niveles 3 y 4. La especificación guía la IA y define el ownership. La IA **ejecuta bajo gobierno**.

© 2026 NTT DATA, Inc.

Equipo NTT DATA — Formadores del programa

Quienes *te acompañan.*

FORMADORES



Jonatan Moreno

Technical Trainer · Agentic Engineering

IA Expert Architect, +10 años en software y arquitectura. Trainer Técnico · Skills/MCP/SDD e integración con workflows.

.



Yefrin Laura

Technical Trainer · IA Architect

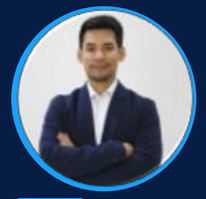
IA Architect. + 8 años en software y arquitectura frontend. Especialista en aplicación IA Engineering en SDLC.



Adolfo Julcamoro

Technical Trainer · Expert Architect

Expert Architect, +10 años en software y arquitectura. Agentic IA, IT Infrastructure Mgmnt, DevOps, Cloud Computing.



David Osorio

Technical Trainer · AI Engineer

AI Engineer, +5 años en transformación digital en SDLC. Innovación en IA y Blockchain

LÍDERES DE SOPORTE



Israel Bravo

Director de Autonomous & Intelligent Technology (AIT)

+15 años en proyectos y servicios de transformación digital e ingeniería de IA. Liderazgo del programa.

© 2026 NTT DATA, Inc.

Recorrido completo — Vista del programa

Ruta de aprendizaje, *5 días.*

Cinco jornadas de 09:00 a 18:00. Examen de entrada el Día 1; Gate G1 y Gate G2 al cierre de los Días 2 y 4.

1 Día

Prompting & Secure AI Engineering

- Prompting + Context Engineering
- Secure AI Engineering
- Optimización + Refactor
- Docs + Unit Testing

EXAMEN DE ENTRADA
Día 1 · 09:30–10:00

2 Día

Agent Skills, Templates & MCP

- Anatomía del Agent Skill
- Templates del nuevo SDLC
- Tool Calling
- MCP aplicado al SDLC

GATE G1 · EVALUACIÓN
Día 2 · 17:30–18:00

3 Día

SDD + Agentes

- OpenSpec, Spec Kit y Constitution
- Specify → Plan → Tasks → Validate
- Fundamentos de agentes
- Routing, Evaluator y HITL

4 Día

Agentic Engineering in Practice

- CrewAI: agentes por roles
- LangGraph: state y routing
- Harness Engineering
- Guardrails

GATE G2 · EVALUACIÓN
Día 4 · 17:30–18:00

5 Día

Hackathon AI Engineering

- Alinear y priorizar
- Construir en dos sprints
- Validar y congelar la versión
- Demo Arena y certificación

✓ CERTIFICACIÓN
Día 5 · 17:30–18:00

© 2026 NTT DATA, Inc.

MODELO DE APRENDIZAJE — cómo se fija lo aprendido

La regla *70 / 20 / 10.*

La adquisición técnica con IA sigue el mismo principio que cualquier oficio: **se aprende haciendo**, mirando a otros hacerlo y reflexionando sobre lo hecho — no mirando un vídeo.

70%
HACIENDO

Practicar en pares, romper prompts, leer código revisado.

El grueso del aprendizaje pasa en los 40 - 60 minutos de práctica y en el código que el trainer se lleva para usar con sus equipos.

20%
Evidencias

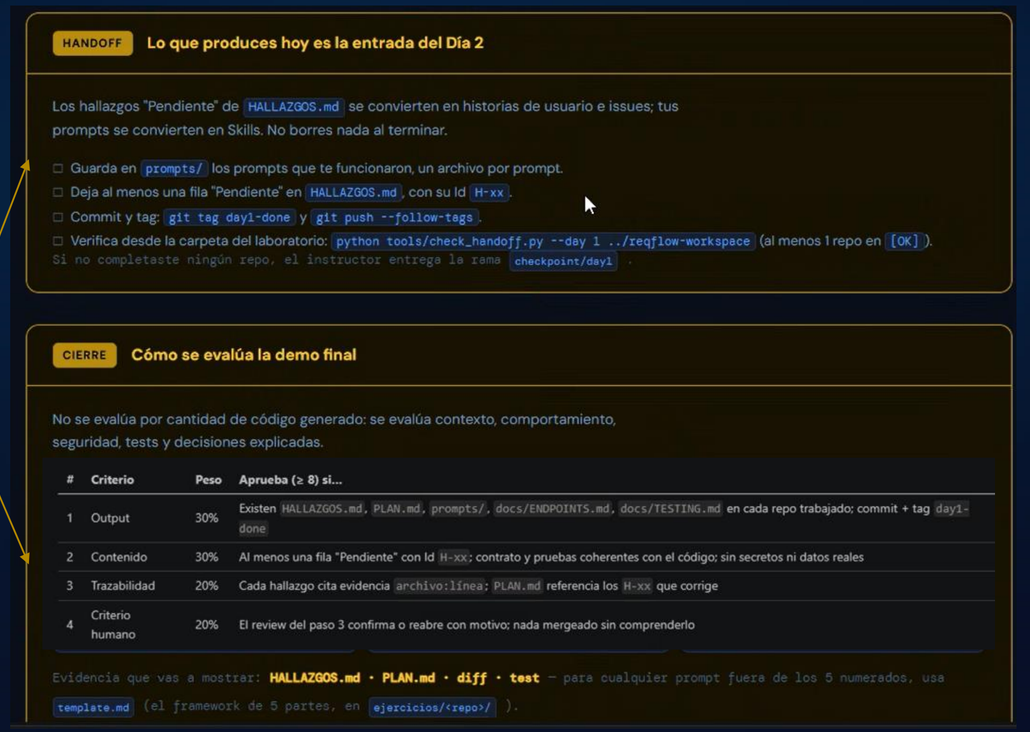
Ver al trainer demostrando y leer patrones reales.

Las demos en vivo contrastan la teoría con casos reales; las réplicas del trainer se vuelven ejemplos después.

10%
LEYENDO

Leer teoría y volver sobre conceptos clave.

Los 20-30 minutos de teoría abren cada sesión; el resto del deck profundiza y referencia.



Si un módulo no tiene práctica, no es formación — es venta.

Programa — Horario por jornada

Cinco jornadas, *un mismo ritmo.*

	DÍA 1 Prompting & Secure AI	DÍA 2 Skills, Templates & MCP	DÍA 3 SDD + Agentes	DÍA 4 Agentic Engineering	DÍA 5 Hackathon E2E
08:45	08:45–09:00 Coffee break	08:45–09:00 Coffee break	08:45–09:00 Coffee break	08:45–09:00 Coffee break	08:45–09:00 Coffee break y registro
09:00	09:00–09:15 Introducción BCP	09:00–09:15 Apertura y repaso Día 1	09:00–09:15 Apertura y repaso Día 2	09:00–09:15 Apertura y repaso Día 3	09:00–09:15 Kick-off: reglas y reto
	09:15–09:45 **Examen de entrada**	09:15–10:00 S1 · Anatomía del Agent Skill	09:15–10:00 S1 · SDD: OpenSpec, Spec Kit	09:15–10:00 S1 · CrewAI: agentes por roles	09:15–09:45 1 · Alinear
10:00	09:45–10:30 M1 · Prompting + Context Eng.	10:00–10:50 Lab 1 · Skill básico versionado	10:00–10:50 Lab 1 · Definición técnica + contrato	10:00–11:00 Lab 1 · Crew API / BE / FE / reviewer	09:45–10:15 2 · Priorizar
	10:30–11:15 Lab 1 · Reescribir pedidos vagos	10:50–11:30 S2 · Templates del SDLC	10:50–11:30 S2 · Specify → Plan → Tasks	11:00–11:35 S2 · LangGraph: state y routing	10:15–10:30 Checkpoint 1 · Scope
11:00	11:15–11:50 M2 · Secure AI Engineering	11:30–12:30 Lab 2 · Skill desde template	11:30–12:30 Lab 2 · Spec-to-tasks	11:35–12:30 Lab 2 · Graph con estado	10:30–12:30 3 · Construir · Sprint 1
12:00	11:50–12:30 Lab 2 · Detección y corrección segura				
13:00	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo	12:30–13:30 Almuerzo
14:00	13:30–14:15 M3 · Refactor y optimización + demo	13:30–14:20 S3 · Tool calling + demo	13:30–14:15 S3 · Fundamentos de agentes	13:30–14:00 S3 · Harness Engineering	13:30–14:45 3 · Construir · Sprint 2
	14:15–15:20 Lab 3 · Refactor guiado con Copilot	14:20–15:20 S4 · MCP: tools, resources y permisos	14:15–15:25 Lab 3 · Single-agent	14:00–15:15 Lab 3 · Harness del workflow	14:45–15:30 4 · Validar
15:00	15:20–16:00 M4 · Docs + unit testing asistido	15:20–16:00 Lab 3 · Skill + MCP en local	15:25–16:00 S4 · Routing, Evaluator y HITL	15:15–16:00 S4 · Guardrails	15:30–16:00 Checkpoint 2 · Freeze
16:00	16:00–16:15 Coffee break	16:00–16:15 Coffee break	16:00–16:15 Coffee break	16:00–16:15 Coffee break	16:00–16:15 Coffee break
	16:15–17:45 Lab 4 · Docs y unit tests	16:15–17:15 Lab 3 (cont.) · Skill + MCP	16:15–17:45 Lab 4 · Agente + review gate	16:15–17:10 Lab 4 · Agente seguro	16:15–17:15 5 · Demo Arena · 2 salas × 5 equipos
17:00		17:15–17:30 Cierre del Día 2		17:10–17:30 Síntesis · puente al Día 5	17:15–17:30 Deliberación del jurado
	17:45–18:00 Cierre · puente al Día 2	17:30–18:00 **Gate G1 · Evaluación**	17:45–18:00 Cierre · puente al Día 4	17:30–18:00 **Gate G2 · Evaluación**	17:30–18:00 **Cierre + Certificado**
18:00					

© 2026 NTT DATA, Inc.

Contenido · Laboratorio · Apertura · cierre · Examen y Gates G1 / G2 · Certificación · Coffee · almuerzo

DÍA 1 · FORMACIÓN IA

Prompting & Secure *AI Engineering.*



Fundamentos para construir software con IA de forma segura, verificable y escalable: prompting, contexto, seguridad, refactor, documentación y tests.

AUDIENCIA	DURACIÓN	FORMATO
Trainers internos BCP	**4 módulos × 1 hora**	**Teoría · Demo · Práctica**

© 2026 NTT DATA, Inc.

DÍA 1 — Objetivos y recorrido

Cuatro paradas de una hora *cada una.*

Terminar el día significa poder **diseñar prompts verificables, escribir código con IA bajo control, detectar lo inseguro y entregar evidencia de calidad**.

S1

Prompting + Context Engineering

Construir instrucciones verificables y dar contexto útil antes de pedirle nada al modelo.

S2

Secure AI Engineering

Detectar lo que la IA genera con riesgo y aplicar controles antes de confiar.

S3

Optimización + Refactor

Reducir complejidad, mejorar mantenibilidad y preservar el comportamiento.

S4

Docs + Unit Testing — cerrar el día con evidencia.

PROMESA DEL DÍA

4 Ses.

Cuatro paradas
Cada una cierra con práctica.

Quien termina el día tiene un cuaderno de patrones, un checklist de seguridad y los cuatro archivos de proyecto.

Cada módulo se sostiene solo, pero juntos forman la base del programa completo.

© 2026 NTT DATA, Inc.

MÓDULO 1 — Prompting & Context Engineering

S1

Pedirle bien a la IA *es la base*.

Antes de pedirle a la IA que construya cualquier cosa — un endpoint, un test, una migración — hay que aprender a **formular, restringir y dejar a la IA con el menor espacio posible para inventar**.

Duración estimada · 60 minutos · trainer al frente, equipo trabajando en pares

¿Qué es *realmente* un prompt?

No es magia, no es un comando arcano ni una frase que "activa" algo. Es lo que le escribes a la IA para pedirle algo **igual que le escribirías a una persona nueva en el equipo**. Funciona como una conversación: pides, miras la respuesta, vuelves a pedir mejor.

EJEMPLO VAGO

"Por favor, hazme una lista de tareas para cumplir el objetivo de hoy de mi trabajo y que sea ordenada."

¿Qué pasa? La IA tiene que inventar casi todo.

EJEMPLO ÚTIL

"Necesito un endpoint POST /api/auth/login que valide email y password con zod, devuelva un JWT firmando {sub:userId} y responda 401 ante error."

Qué pasa · la IA ya sabe qué tiene que construir.

LO QUE NO ES

Una sola frase mágica.

No es **una sola frase perfecta**.
No es **un comando con sintaxis secreta**.

LO QUE SÍ ES

Una conversación de varios intentos.

Pides, revisas, vuelves a pedir mejor. **Cada iteración reduce lo que la IA tiene que inventar** y deja solo lo que tú puedes respaldar.

© 2026 NTT DATA, Inc.

Cinco elementos, *uno por bloque.*

Piensa cada bloque del prompt como la información que le darías a una persona nueva antes de pedirle la tarea. Cada elemento **recorta el espacio para inventar**.

01

Contexto

¿Qué necesita saber antes? Stack, archivos, dominio del problema, restricciones de negocio.

02

Objetivo

Una sola frase: qué quiero lograr. Bien, no bien, listo cuándo.

03

Restricciones

Lo que NO se puede romper. Endpoints, contratos, dependencias, decisiones ya tomadas.

04

Criterios

Cómo sabré que quedó. Contratos respetados, tests pasando, comportamiento previo.

05

Formato de salida

Cómo espero que vuelva: lista, plan, diff, archivo. Opcional pero reduce malentendidos.

DE LO GENERAL A LO CONCRETO

El orden importa *y empieza por el contexto.*

El contexto define el resto. Sin él, los objetivos se vuelven vagos, las restricciones se adivinan y los criterios se vuelven wishful thinking. Una práctica útil: cuando **un prompt falla**, revisa primero el bloque de contexto antes que todo lo demás.

Un buen prompt es un contrato pequeño
que firma la IA contigo.

Define qué necesita saber.
Define cómo debe responder.
Define qué no puede romper.

El formato no es obligatorio pero reduce malentendidos — sobre todo entre equipos.

De vago a verificable *en un solo movimiento.*

ANTES

"Hazme un login. Rápido,
para hoy, sale a producción mañana."

Qué pasa detrás:

La IA no sabe qué tipo de auth pedir.
No sabe qué errores mostrar.
No sabe si guarda token o cookie.
No sabe si hay rate limit o no.
Resuelve inventando, sin avisar.

Resultado · a revisar casi todo.

DESPUÉS

"Endpoint POST /api/auth/login que valide
body {email, password},
persista al usuario vía userRepo.findByEmail, compare hash
Con bcrypt, devuelva JWT firmado con {sub:userId}
y responda 401 si las credenciales no coinciden."

Qué cambió:

Verbo + ruta. La IA ya sabe qué construir.
Validación especificada — fuente de verdad única.
Método de comparación y librería declaradas.
Forma del token y claim principal definidos.
Respuesta de error codificada.

Resultado · primera iteración útil.

La diferencia no es pedir más — es **dejar menos huecos**.

Cuatro patrones *que se repiten.*

Los patrones son **piezas reutilizables** que la IA responde mejor cuando se las das en el prompt. Útiles en código, en tests y en docs.

01

Rol — quién debe responder

Decirle "Actúa como un senior de TypeScript con 10 años en fintech" cambia el tono, el rigor y los silencios que la IA elige. Inicia el prompt con un rol — útil en casi todo.

02

Few-shot — ejemplos que enseñan

Mostrarle dos o tres ejemplos de input → output esperado y la IA aprende el patrón. Más rápido y más estable que describir la forma en prosa.

03

Cadena de pensamiento

Para problemas con varios pasos, pedirle que piense primero en voz alta ("piensa paso a paso antes de responder") y luego dé la respuesta final. Reduce errores tontos.

04

Restricciones explícitas

"No cambies los nombres públicos", "no agregues dependencias", "no asumas lo que no está documentado". Una restricción bien puesta vale por cinco descripciones.

Cuándo combinar y cuándo no

Los patrones se **combinan**. Un caso real puede ser: rol + few-shot + restricciones + cadena de pensamiento, en ese orden. Pero **no hace falta usar los cuatro siempre** — depende del tamaño y la claridad del cambio pedido.

Seis antipatrones *que vemos cada semana.*

Salen de la observación de pares de trainers que entregaron prompts y no podían explicar por qué no funcionaron. No son listas exhaustivas — son las más frecuentes.

Pedir con prisa

"Hazlo rápido, hoy sale a producción". La IA interpreta velocidad como libertad de inventar.

Asumir archivos invisibles

"Arregla esto" sin decir qué hay, qué versión, qué framework. La IA reescribe todo.

Perder la pregunta en respuestas largas

No poner criterios — la IA vuelve con cinco párrafos y la duda viva.

Pedir TODO a la vez

Diez cosas en un solo prompt. La IA resuelve tres, ignora cuatro e improvisa tres.

Confiar en el tono

Porque "suena seguro" no significa que lo sea. Validar siempre.

Pegar código real sin anonimizar

Secrets, tokens, datos de cliente — todo eso entra al modelo. Tema del Módulo 2.

El costo de un mal prompt *siempre lo paga el siguiente paso.*

El antipatron más caro es dar por buena una primera respuesta. La próxima iteración sale más cara que la primera — recuperar el control toma más esfuerzo que mantenerlo desde el principio. Por eso cada bloque del framework cuenta.

Si suena a "demasiado simple, quizá funcione", toca revisar.

Lab 1 — Reescribir *un pedido vago.*

En parejas, transformar **tres pedidos vagos** en tres prompts con los cinco elementos. Sin generar código todavía — sólo el prompt en papel o en un archivo compartido.

PEDIDO 1 · BACKEND

"Necesito un endpoint que haga algo con usuarios y roles."

Reescribir con 5 elementos

PEDIDO 2 · DOCUMENTACIÓN

"Documenta este módulo para que cualquiera lo entienda."

Reescribir con 5 elementos

PEDIDO 3 · TESTS

"Haz tests al servicio de pagos."

Reescribir con 5 elementos

Cómo se hace

Tres prompts reescritos, uno por tarjeta.
Subirlos al chat compartido del equipo.
El trainer pasa y comenta tres.
Discutir qué cambió en común.

Evidencia

Tres pairs · un prompt por pedido · foto del cuaderno o archivo prompts.txt en el chat.

Sin código todavía — esto es sobre la conversación, no sobre la respuesta.

MÓDULO 2 — Secure AI Engineering

S2

No dejar que la velocidad *coma a la pregunta.*

La IA ejecuta rápido y obedece sin dudar. La pregunta ¿esto es seguro? tiene que seguir existiendo — antes, durante y después del código generado.

Duración estimada · 60 minutos · trainer con énfasis en revisión humana

La IA es un empleado nuevo *muy rápido.*

Hace **inocentemente** lo que le pides, sin tener criterio propio. Su poder no viene de entender — viene de lo rápido que produce. **La responsabilidad del criterio sigue siendo tuya.**

Si pides una puerta, obtendrás una puerta.

Pero si no especificas que venga con cerradura, **se construirá sin cerradura**. Y no lo hará con mala intención — lo hará porque no sabía. El daño no es malicia, es ausencia de criterio. Y por eso la respuesta segura **no es suficiente**.

Piénsalo como un junior sin conocimiento del proyecto — y que no se frustra si le haces repetir.

LO QUE IMPLICA

Tres consecuencias

Especificar antes.
Revisar después.
No firmar lo que no sepas leer.

!

El daño silencioso — el que más cuesta

La IA no falla ruidosamente cuando algo es inseguro. Falla detrás de una fachada razonable: tests que pasan, código que se ve serio, una sonrisa en pantalla. Por eso **la revisión humana nunca es opcional**. Es el control que mantiene al sistema bajo control.

Entonces, si no podemos confiar, *qué preguntamos.*

Antes de confiar, ejecutar o mergear — **cinco preguntas** ordenadas mentales. Las mismas cada vez, en cualquier módulo.

PREGUNTA 1

¿La IA generó código vulnerable?

Buscar vulnerabilidades conocidas: XSS, SQLi, SSRF, deserialización, hard-coded secrets, eval, f-string con datos del usuario.

PREGUNTA 2

¿Se ejecuta código o SQL directo?

Si es sí — asegurar prepared statements, ORM, validación, sandboxing. Si eval o child_process — quitarlo de inmediato.

PREGUNTA 3

¿Faltan chequeos de autorización?

Un endpoint sin authz no es un endpoint — es una fuga con vestido de endpoint.

PREGUNTA 4

¿Le pasé al prompt información sensible de la empresa o del cliente?

Si es sí — el archivo REGLAS.md del Módulo 1 aplica acá también. Antes de pegar código real con tokens, nombres de cliente o endpoints internos, sanitizar y referenciar.

PREGUNTA 5

¿Quién revisa este código antes de producción?

Si la respuesta no es clara — todavía no está listo.

Estas preguntas **no se firman al final del módulo** — se firman en la cabeza del trainer antes de mergear cualquier cosa.

Cinco patrones inseguros *típicos.*

Salen una y otra vez en diffs revisados por humanos. Conocerlos cambia la conversación — **de revisarlos caso a caso a detectarlos en cinco segundos**.

01 Input sin validar

Formularios, query params y bodies sin schema. La IA construye al endpoint más simple posible — sin validar lo que ingresa.

02 Secretos en el código

API keys, tokens, passwords pegados al código. La IA los toma de ejemplos abiertos — y los pega.

03 Permisos demasiado amplios

Permisos que la IA pide "para que sea genérico": tokens con scope de escritura, cuentas de servicio con admin.

04 Funciones en el lugar equivocado

Tareas pesadas en el hilo del request, endpoints públicos con demasiada lógica, acceso a filesystem o ejecución sin sandbox.

05 Consultas a BD construidas mal

Concatenación de strings en queries SQL. ORM bien usado — concatenación y f-strings directos — fuga esperando.

Cinco patrones, una sola respuesta

Cuando detectes uno, la conversación es con la IA — no con el sistema. Lo que llamas patrón inseguro no es contravenir una regla: es **una forma de razonar sobre el código que está por generarse**. Una vez que el patrón se vuelve vocabulario del trainer, la revisión se vuelve hábito.

Defensa en *profundidad.*

No hay una sola práctica que asegure la seguridad — lo que sirve es combinar: **un archivo de reglas, una pasada de revisión y un humano que firma**.

PASO 1 · ANTES

Transcribir las reglas a un archivo

`REGLAS.md`

Pega en un solo archivo las reglas de seguridad, los secretos que nunca deben aparecer y los contratos que no se rompen. Listo para referenciar.

PASO 2 · AL CONSTRUIR

Mencionar REGLAS.md en el prompt

"Construye el CRUD hacia tabla usuario considerando el archivo REGLAS.md".

La IA lee antes — el archivo hace el trabajo pesado y evita llamadas redundantes.

PASO 3 · DESPUÉS

Pasada de auditoría con IA

"Revisa el backend, todos los endpoints, audita contra REGLAS.md y resuelve las vulnerabilidades encontradas".

Y sobre todo — **un humano revisa antes de mergear**. La IA no es auditor final.

Y entonces, ¿quién revisa?

La respuesta corta es: **siempre un humano, antes de mergear**. Las tres prácticas (reglas, mención, auditoría) son *complementos*, no reemplazos. La IA no firma nada, no aprueba nada, no sabe lo que es BCP. Firmar sigue siendo del trainer y del equipo que recibe.

Tres capas que se suman — humano que firma va al final.

El caso del backend de *usuarios.*

Un prompt que sigue los pasos del slide anterior: reglas primero, código segundo, auditoría después. Compáralo con el ejemplo vago del Módulo 1.

PASO 1 · AL CONSTRUIR

*"**Construye** un backend seguro de usuarios, considerando el archivo **REGLAS.md**.*
*El backend debe ser un **CRUD** contra la tabla **usuario**:*
validación con zod, password con bcrypt,
errores controlados y sin secretos en logs."

PASO 2 · DESPUÉS DE CONSTRUIR

*"**Revisa** el backend detalladamente, todos los endpoints. Audita el código generado contra **REGLAS.md** y resuelve las **vulnerabilidades** encontradas."*

Por qué este prompt *funciona mejor*

Tiene el archivo de reglas como entrada — la IA no tiene que adivinar límites.
Mención explícita de validación, hashing y errores — los típicos patrones inseguros cerrados por anticipado.
Una segunda pasada contra el mismo archivo — los resabios los recoge la auditoría.

Un prompt que vale por tres — antes, durante y después.

Lab 2 — Red team *al prompt.*

En parejas, tomar el prompt del slide anterior y **atacarlo**. ¿Qué puede pasar si se manda así? Discutir vulnerabilidades primero, defensas después.

PRIMERA PASADA · BUSCAR FISURAS

¿Qué falta en este prompt?

Logging de requests sensibles.
Limitación de intentos por IP o usuario.
Manejo de errores sin filtrar stack.
Verificación de propiedad del recurso.

SEGUNDA PASADA · ENDURECER

¿Cómo cierra cada hueco?

Añadir restricciones al **REGLAS.md** — la auditoría posterior las lee del mismo archivo. Cada gap que encuentran se agrega al archivo.

ENTREGABLE

Un par de prompts · antes y después

El prompt inicial del Lab 2 con anotaciones de huecos.
Una versión endurecida que cierra los huecos.
Cuatro hallazgos priorizados en una lista corta.

25 minutos · trainer pasa, comenta tres hallazgos en común

MÓDULO 3 — Optimización + Refactor

S3

Mejorar el código *sin romper producción.*

El arte de **reducir la complejidad preservando el comportamiento** del sistema — y potenciarlo.

Duración estimada · 60 minutos

MÓDULO 3 — dos palabras que se confunden

Refactor no es optimizar *(y viceversa).*

Antes de pedirle nada a la IA, el trainer debe poder decir **qué tipo de cambio** está pidiendo. Si no lo nombras, la IA mezcla los dos — y ahí aparece el ruido.

REFACTOR

Cambia la forma, no el comportamiento.

Reducir duplicación y nombres confusos.
Separar responsabilidades.
Mismos inputs → mismos outputs.
Se valida con los tests existentes.

OPTIMIZACIÓN

Mejora una métrica concreta.

Menos queries innecesarias.
Menor latencia o consumo de memoria.
Necesita una medición antes y después.
Sin métrica, no es optimización.

Regla práctica para el aula: un prompt = un tipo de cambio. Primero refactor con tests verdes; luego optimización con métrica medida.

© 2026 NTT DATA, Inc.

Tres piezas clave *antes de tocar código.*

Permiten que la IA proponga mejoras **útiles**, **alineadas al proyecto** y **verificadas**. Si falta una, el refactor se vuelve apuesta.

01

Criterio

Qué significa "mejor" en este proyecto: legibilidad, menos duplicación, menos queries. **Lo decide el equipo, no la IA.**

Pregunta: ¿qué quiero lograr y qué no?

02

Contexto

Contratos de API, buenas prácticas y reglas documentadas en archivos que la IA consulta: **ENDPOINTS.md, GOOD_PRACTICES.md, RULES.md.**

Pregunta: ¿qué no puede romper?

03

Pruebas

Tests que corren **antes y después** del cambio. Son la única forma de demostrar que el comportamiento se preservó.

Pregunta: ¿cómo sabré que funciona?

Sin tests previos, el primer paso no es refactorizar: es **pedirle a la IA que escriba tests de caracterización** del comportamiento actual.

© 2026 NTT DATA, Inc.

La IA mejora el código, *y también crea ruido.*

Sucede muchas veces en la vida real, cuando la IA no tiene suficiente información. Cinco señales para revisar en cada diff:

Cambia lógica sin avisar

Modifica comportamiento innecesariamente para lograr su objetivo.

Control: pedir el diff explicado línea a línea.

Sobre-abstrae

Crea factories, interfaces o helpers innecesarios.

Control: restricción "no agregues capas nuevas".

Rompe contratos

Modifica DTOs, nombres o respuestas que usa frontend o backend.

Control: ENDPOINTS.md como fuente de verdad.

Falsos positivos

Te dirá que terminó "estupendamente refactorizado", pero no lo sabe: solo infiere.

Control: la evidencia es la corrida de tests, no el mensaje.

Importaciones faltantes

Al mover código entre archivos, olvida importar lo necesario.

Control: compilar, lint y build antes de aceptar.

Para el trainer: muestra un diff real con al menos dos de estas señales y pide al grupo que las encuentre antes de explicarlas.

© 2026 NTT DATA, Inc.

Un refactor seguro *se pide en dos fases.*

FASE 1 · ANALIZAR → PLAN.md

Analiza este código antes de modificarlo.

Contexto · consulta primero ENDPOINTS.md, GOOD_PRACTICES.md y RULES.md.

Objetivos · reducir duplicación, mejorar legibilidad, evitar queries innecesarias, mantener el comportamiento actual.

Restricciones · no cambies endpoints, DTOs ni reglas de negocio; no agregues dependencias; no asumas cambios no documentados.

Criterios · contratos intactos, reglas respetadas, tests en verde.

Respuesta · genera PLAN.md con problemas, cambios propuestos y archivos a modificar. **No modifiques código todavía.**

FASE 2 · IMPLEMENTAR EL PLAN

Lee primero PLAN.md y úsalo como guía.

- Implementa únicamente lo pendiente.
- Respeta ENDPOINTS.md, GOOD_PRACTICES.md y RULES.md.
- No agregues cambios fuera del plan.
- Al finalizar, explica el diff y cualquier desviación.

Por qué dos fases

Entre fase 1 y fase 2 hay un **punto de control humano**: el trainer revisa y corrige PLAN.md antes de que la IA escriba una sola línea.

© 2026 NTT DATA, Inc.

MODULO 3 · LABORATORIO

Lab 3 — Refactor guiado *con PLAN.md.*

Sobre un servicio con duplicación visible (por ejemplo, un CRUD con tres endpoints que repiten validación), aplicar el flujo de dos fases y demostrar que nada se rompió.

1

Correr tests

Guardar el resultado inicial. Si no hay tests, pedir tests de caracterización.

2

Fase 1 · PLAN.md

Usar el prompt de análisis. Revisar el plan en pareja y tachar lo que sobra.

3

Fase 2 · implementar

Ejecutar solo el plan aprobado. Leer el diff completo.

4

Verificar

Tests otra vez en verde, build y lint. Revisar las cinco señales de ruido.

EVIDENCIA

PLAN.md revisado · diff final · captura de tests antes y después · una señal de ruido detectada y cómo se corrigió.

© 2026 NTT DATA, Inc.

MÓDULO 4 — Docs + Unit Testing

S4

Cerrar el día *con evidencia.*

Generar documentación útil y pruebas que **demuestren calidad y funcionamiento** — no que solo lo aparenten.

Duración estimada · 60 minutos

MÓDULO 4 — para qué sirve documentar

Documentar no es llenar *un Markdown.*

Explica **qué hace el código, cómo usarlo y qué reglas debes respetar al modificarlo**. Tres preguntas ordenan cualquier documento.

¿QUÉ DEBE EXPLICAR?

Qué hace el módulo, **cómo se usa**, qué contratos respeta y qué no debe cambiar.

¿PARA QUIÉN SIRVE?

Developers, reviewers, QA y agentes de IA que deben entender el proyecto antes de tocarlo.

¿QUÉ EVITA?

Cambios **innecesarios**, refactors que **rompen** y pruebas sin relación con el negocio.

> ***"Si la documentación no ayuda a tomar una decisión o validar un cambio, probablemente sobra."***

© 2026 NTT DATA, Inc.

Cuatro archivos *que realmente ayudan.*

Pocos archivos, cada uno con una responsabilidad clara. La IA puede redactarlos; **la revisión y corrección son humanas**. Son los mismos que usamos en S2 y S3.

Archivo	Qué contiene	Dónde se usa
`ENDPOINTS.md`	Contratos, inputs, outputs y códigos de respuesta.	Refactor sin romper contratos (S3)
`RULES.md`	Reglas o lineamientos que no deben romperse.	Construcción y auditoría segura (S2)
`GOOD_PRACTICES.md`	Convenciones técnicas y decisiones del equipo.	Criterio de "mejor código" (S3)
`TESTING.md`	Qué se prueba, cómo correrlo y qué casos son críticos.	Generación de tests útiles (S4)

Ilación del día: lo que documentas en S4 es el contexto que alimenta los prompts de S1, la seguridad de S2 y el refactor de S3. El kit se reutiliza en el Día 2 como base de los Agent Skills.

© 2026 NTT DATA, Inc.

Cuatro errores típicos *con Docs y Tests.*

Documentación que miente

Describe endpoints que no coinciden con el contrato real o están desactualizados.

Antídoto: generar la doc desde el código y contrastarla con ENDPOINTS.md.

Tests que solo cubren lo fácil

Mucho coverage, pero sin probar lo que de verdad importa al negocio.

Antídoto: listar casos críticos en TESTING.md antes de pedir tests.

Tests adaptados al bug

La IA ajusta el test para que pase, en lugar de probar el problema real.

Antídoto: restricción "no modifiques tests existentes sin justificarlo".

Backend y frontend no se entienden

DTOs y modelos de ambos proyectos no conversan: son diferentes.

Antídoto: un solo contrato compartido como fuente de verdad.

Pregunta para el grupo: ¿cuál de los cuatro han visto esta semana en su propio repositorio?

© 2026 NTT DATA, Inc.

MÓDULO 4 — Loop Engineering

Trabajar en ciclos *hasta cumplir el criterio.*

Loop Engineering es trabajar en **ciclos de planificación, ejecución, prueba y corrección** hasta completar las tareas con criterios de aceptación definidos.

1

Objetivo

¿Qué quieres lograr? Escrito como criterio de aceptación.

2

Implementación

Dividir en tareas dentro de PLAN.md y desarrollar.

3

Revisión

Correr tests: ¿cumple los requisitos?

4

Iteración

Corregir y repetir hasta alcanzar el objetivo.

PROMPT DE LOOP

"Lee PLAN.md y TESTING.md. Itera el proceso de resolución de tareas hasta completarlas todas o conseguir el objetivo. Tras cada tarea, corre los tests; no modifiques un test para que pase sin explicarlo."

© 2026 NTT DATA, Inc.

MÓDULO 4 · LABORATORIO

Lab 4 — Docs y tests *que demuestran.*

Sobre el servicio refactorizado en el Lab 3: generar el kit de documentación y una batería de tests que cubra los casos críticos del negocio.

PARTE A · DOCUMENTAR

Pedir a la IA ENDPOINTS.md y TESTING.md a partir del código.
Revisar en pareja: ¿la doc miente en algún punto?
Corregir a mano y marcar lo que cambió.

PARTE B · PROBAR

Listar tres casos críticos y dos casos de error.
Pedir tests con el prompt de loop.
Introducir un bug a propósito: ¿algún test lo detecta?

EVIDENCIA DE CIERRE DEL DÍA

Kit de 4 archivos revisado · suite de tests en verde · prueba del bug intencional detectado por al menos un test.

© 2026 NTT DATA, Inc.

CIERRE DEL DÍA 1 — lo que te llevas

Contextualizar, pedir, revisar, *probar y decidir.*

Cuatro módulos, una sola idea: **usar IA sin perder el control de la ingeniería**. Cada sesión deja una evidencia concreta.

Módulo	Idea clave	Evidencia
S1 · Prompting + Contexto	Cinco elementos dejan menos espacio para inventar.	Tres prompts reescritos
S2 · Secure AI	REGLAS.md antes, auditoría después, humano que firma.	Prompt endurecido + hallazgos
S3 · Optimización + Refactor	Criterio, contexto y pruebas; dos fases con PLAN.md.	PLAN.md + diff + tests
S4 · Docs + Unit Testing	Documentar para decidir; tests que detectan bugs reales.	Kit de 4 archivos + suite

PUENTE AL DÍA 2

El kit de archivos de hoy es la materia prima de **Dia 2**: Agent Skills, templates base para SDLC con IA, Tool Calling y MCP.

© 2026 NTT DATA, Inc.


NTT DATA